

Technische Hochschule Augsburg
An der Hochschule 1
D-86161 Augsburg



01.02.2026

Integration der RISC-V M-Extension in den PicoNut-Prozessor

Projektarbeit 2

Tristan Kundrat

Matrikelnummer: 0000000

Fakultät:	Informatik
Studiengang:	Technische Informatik
Semester:	WiSe2025/26
Kurs:	PicoNut: Ein modularer RISC-V-Prozessor
Prüfer/in:	Prof. Dr. Gundolf Kiefer
Abgabedatum:	01.02.2026

Lizenzen

© Tristan Kundrat 2026

Diese Arbeit mit dem Titel

Integration der RISC-V M-Extension in den PicoNut-Prozessor

von Tristan Kundrat steht unter folgender Lizenz:

Creative Commons Namensnennung-Nicht kommerziell-Share Alike 4.0 International



<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.de>

Sämtliche, in der Arbeit beschriebene und auf dem beigelegten Datenträger vorhandene, Ergebnisse dieser Arbeit in Form von Quelltexten, Software und Konzeptentwürfen stehen unter einer

GNU General Public License Version 3

<http://www.gnu.de/documents/gpl.de.html>

Die Typst-Vorlage dieser Arbeit unter folgendem Repository:

<https://github.com/fuchs-fabian/typst-template-aio-studi-and-thesis>

wurde erstellt von © Fabian Fuchs und unterliegt der

MIT License

<https://mit-license.org/>

Erklärung zur Projektarbeit

Ich, Tristan Kundrat (Matrikelnummer 0000000), versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema

Integration der RISC-V M-Extension in den PicoNut-Prozessor

selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, wobei ich alle wörtlichen und sinngemäßen Zitate als solche gekennzeichnet habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Das Merkblatt zum Täuschungsverbot im Prüfungsverfahren der Technischen Hochschule Augsburg habe ich gelesen und zur Kenntnis genommen. Ich versichere, dass die von mir abgegebene Arbeit keinerlei Plagiate, Texte oder Bilder umfasst, die durch von mir beauftragte Dritte erstellt wurden.

Ich bin mir bewusst, dass mit KI generierte Texte keine Garantie für die Qualität von Inhalten und Text bieten. Daher erkläre ich, dass ich KI-Werkzeuge lediglich als Hilfsmittel genutzt habe, die von KI generierten Inhalte kritisch überprüft habe und mein eigenständiger kognitiver sowie kreativer Einfluss in dieser Arbeit überwiegt. Ich versichere, dass ich die Inhalte meiner Arbeit vollständig verstanden habe und selbstständig vertreten kann.

Emmering, den 01.02.2026.

T. Kundrat

Tristan Kundrat

Inhaltsverzeichnis

LIZENZEN	2
ERKLÄRUNG ZUR PROJEKTARBEIT	2
ABBILDUNGSVERZEICHNIS	II
1. EINLEITUNG	1
2. M-EXTENSION	2
2.1. Anforderungen an die M-Extension	2
2.2. Überblick: PicoNut und Multiplikations-/Divisionsmodul	2
2.3. Integration in den Prozessorkern	5
2.4. Implementierung des Multiplikations- und Divisionsmoduls	7
2.5. Testen des neuen Nucleus mit M-Extension	13
3. FAZIT	14
3.1. Zusammenfassung	14
3.2. Ausblick	14
LITERATUR- UND QUELLENVERZEICHNIS	15

Abbildungsverzeichnis

Abbildung 1	Aufbau des PicoNut-Systems	2
Abbildung 2	Blockdiagramm des Multiplikations-/Divisionsmoduls	3
Abbildung 3	R-Type Instruktion in der RISC-V ISA (Abb.: RISC-V Foundation [1])	4
Abbildung 4	Zustandsübergangsdiagramm des Steuerwerks ohne M-Extension	6
Abbildung 5	Zustandsübergangsdiagramm des Steuerwerks mit M-Extension	7
Abbildung 6	Beispiel der Shift-and-add Berechnung nach Koren [2]	8
Abbildung 7	Zustandsübergangsdiagramm des Multiplikationsmoduls	9
Abbildung 8	Beispiel der Shift-and-subtract Berechnung nach Koren [2]	11
Abbildung 9	Zustandsübergangsdiagramm des Divisionsmoduls	12

1. Einleitung

Das Ziel des PicoNut-Projekts [3] ist die Entwicklung eines minimalen RISC-V-Prozessors, welcher jedoch gleichzeitig flexibel erweiterbar ist.

Der Zweck des Prozessors ist der Einsatz in der Lehre und Forschung. Beispielsweise können die Interaktion zwischen Hard- und Software oder die Entwicklung und Integration von Modulen zur Hardwarebeschleunigung untersucht werden. Die Implementierung des PicoNut-Prozessors basiert dabei auf SystemC [4], einer C++ Klassenbibliothek, die als Hardware-Beschreibungssprache dient.

Die M-Extension der RISC-V-Befehlssatzarchitektur [1] lässt sich als ein solcher Hardwarebeschleuniger realisieren. In dieser Befehlssatzerweiterung werden Prozessorinstruktionen zur Multiplikation, Division und Restberechnung definiert.

Ziel dieser Arbeit ist die vollständige synthesesfähige Implementation aller in der M-Extension definierten Instruktionen in den PicoNut-Prozessor, sodass die dadurch in Hardware unterstützten Rechenoperationen im Vergleich zur Berechnung in Software beschleunigt werden.

Der Aufbau der Arbeit ist wie folgt: Zunächst werden in Abschnitt 2.1 die Anforderungen an die M-Extension erläutert, worauf Abschnitt 2.2 einen Überblick über den PicoNut und die Anschlüsse der Multiplikations- und Divisionsmodule gibt. Im Anschluss wird unter Abschnitt 2.3 die genaue Integration der M-Extension in den PicoNut-Core dargelegt, auf das die tatsächliche Implementation der Multiplikation und Division in Hardware in Abschnitt 2.4 folgt. Zuletzt wird in Abschnitt 2.5 das Testen des neuen Prozessorkerns beschrieben, worauf anschließend unter Abschnitt 3 eine Zusammenfassung der Ergebnisse sowie ein Ausblick auf mögliche Erweiterungen folgt.

Diese schriftliche Ausarbeitung wurde mithilfe von Typst [5] erstellt. Typst ist ein neues Zeichensatzsystem, welches eine Alternative zu bekannten Systemen wie LaTeX oder Google Docs darstellt, wobei insbesondere auf die Kombination von hoher Funktionalität mit Nutzerfreundlichkeit geachtet wird.

2. M-Extension

Alle nachfolgenden Inhalte beziehen sich, sofern nicht anders angegeben, auf die PicoNut-Dokumentation. [6]

2.1. Anforderungen an die M-Extension

Wie schon in der Einleitung erwähnt, ist das Ziel dieser Arbeit, die M-Extension der RISC-V ISA [1] („Instruction set architecture“, deutsch „Befehlssatzarchitektur“) vollständig zu implementieren, sodass der resultierende Prozessorkern synthetisierbar ist.

Im Exposé dieses Projekts wurden konkret folgende Ziele definiert:

- Erweiterung des bestehenden RV32I-Nucleo um die M-Extension
- Modularer Ansatz: Wahl zwischen platzsparender und performanter Recheneinheit möglich
- Einbindung in ALU und Steuerwerk
- Testen der M-Extension in Funktion und Geschwindigkeit

Insbesondere sollen also folgende Instruktionen unterstützt werden:

mul, mulh, mulhu, mulhsu, div, divu, rem, remu

Die genauere Bedeutung der Instruktionen wird in Abschnitt 2.2 erläutert.

Beim modularen Ansatz ist das Vorgehen so zu verstehen, dass innerhalb dieser Projektarbeit nur eine Variante der Berechnung in Hardware implementiert wird. Jedoch wird für die Zukunft die Möglichkeit offen gehalten, andere Varianten zu implementieren, welche ohne erheblichen Aufwand in die bestehende Struktur integriert werden können. Der notwendige Prozess zum Ersetzen der Module muss ebenfalls in der PicoNut-Dokumentation beschrieben werden.

2.2. Überblick: PicoNut und Multiplikations-/Divisionsmodul

Ein PicoNut-System besteht im Wesentlichen aus einem *PicoNut-Core*, dem *Wishbone-Systembus* und gegebenenfalls verschiedenen *Peripheriemodulen*, wie dem Audio- oder UART-Modul.

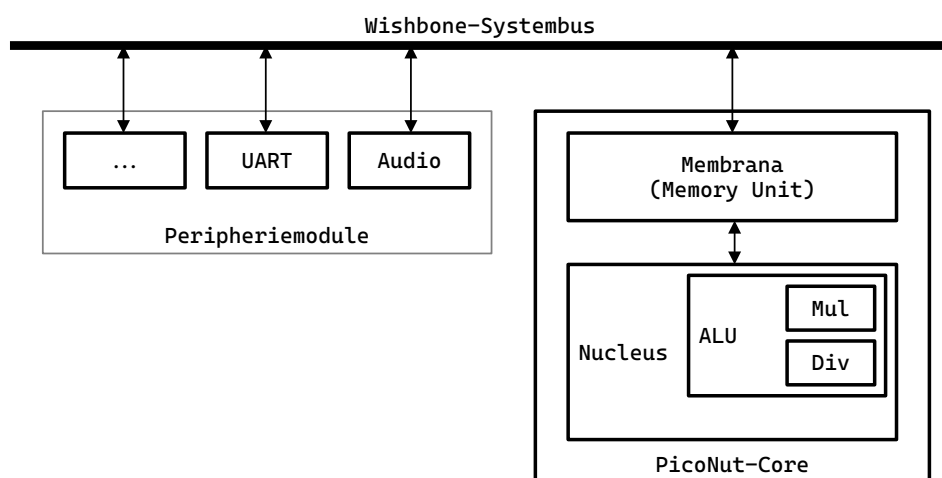


Abbildung 1: **Aufbau des PicoNut-Systems**

In Abbildung 1 wird das Zusammenspiel dieser Komponenten dargestellt. Hier kann man sehen, dass der PicoNut-Core aus zwei Komponenten besteht, der *Membrana* und dem *Nucleus*. Die Membrana (Memory Unit) ist für den Datenaustausch zwischen Nucleus und Wishbone-Systembus zuständig. Im Nucleus werden die eigentlichen Prozessorinstruktionen ausgeführt, die dann von der ALU („Arithmetic logic unit“, deutsch „Arithmetik- und Logikeinheit“) berechnet werden.

Wie schon eingangs in Abschnitt 2.1 erwähnt, soll ein modularer Ansatz gewählt werden. Daher wurde die ALU für die M-Extension um zwei Submodule erweitert, welche fest definierte Anschlüsse besitzen. Die interne Logik zur Berechnung von Multiplikationen oder Divisionen bleibt hierbei abgekapselt und kann ohne weiteres ausgetauscht werden, ohne tiefgreifende Änderungen an der ALU vorzunehmen.

Eines der eben genannten Module ist ein Multiplizierer (Mul im Diagramm) und das andere ein Dividierer (Div im Diagramm).

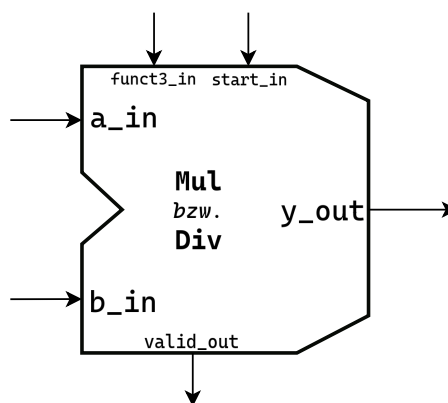


Abbildung 2: **Blockdiagramm des Multiplikations-/Divisionsmoduls**

Wie in Abbildung 2 zu sehen, haben jeweils das Multiplikations- und Divisionsmodul identische Ein- und Ausgangssignale, dessen Bedeutung in Tabelle 1 erläutert werden.

Zusätzlich zu den hier genannten Signalen besitzt das Modul gegebenenfalls noch einen `clk` und `reset` Eingang, falls die Berechnung nicht rein kombinatorisch abläuft. Da diese nur direkt verbunden werden, wurden diese zur Übersichtlichkeit weggelassen.

Signalname	Bitbreite	Beschreibung
<code>a_in</code>	32	Wert für Berechnung
<code>b_in</code>	32	Wert für Berechnung
<code>funct3_in</code>	3	Art der Berechnung, die durchgeführt werden soll
<code>start_in</code>	1	Startet bei 1 die Berechnung
<code>y_out</code>	32	Ergebnis der Berechnung; Abhängig von <code>funct3</code>
<code>valid_out</code>	1	1 bei abgeschlossener Berechnung

Tabelle 1: **Beschreibung der Signale des Multiplikations-/Divisionsmoduls**

Bei `start_in` wird davon ausgegangen, dass alle Eingänge (`a_in`, `b_in`, `funct3_in`) über den gesamten Zeitraum, für den `start_in == 1` gilt, stabil gehalten werden und valide sind.

`funct3_in` ist ein Bestandteil der gerade auszuführenden Instruktion.

In der RISC-V ISA [1] gibt es verschiedene Instruktionstypen mit jeweils unterschiedlichen Inhalten. Alle Instruktionen der M-Extension sind sogenannte *R-Type* Instruktionen, der Aufbau dazu ist in Abbildung 3 dargestellt.

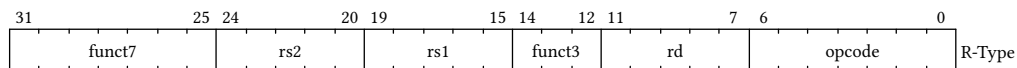


Abbildung 3: **R-Type Instruktion in der RISC-V ISA (Abb.: RISC-V Foundation [1])**

Der *opcode* deutet hierbei mit `0b0110011` an, dass es sich um eine R-Type Instruktion handelt. *rs1* und *rs2* sind die entsprechenden Register, in denen die Eingangswerte für die Berechnung liegen, wobei das Rechenergebnis in Register *rd* geschrieben wird.

In *funct7* und *funct3* wird die eigentliche Rechenoperation codiert. `funct7 == 0b0000001` bedeutet, dass eine Instruktion der M-Extension anliegt. Anhand der drei Bits aus *funct3* unterscheidet der Prozessor zwischen den acht verschiedenen Operationen der M-Extension. Tabelle 2 zeigt die verschiedenen möglichen Werte und deren Bedeutung.

funct3-Wert	Instruktion	Beschreibung
0b000	mul	$rd = (\text{int32_t})(rs1 * rs2)$ rd: untere 32 Bit des Multiplikationsergebnisses
0b001	mulh	$rd = (rs1 * rs2) \gg 32$ rs1: vorzeichenbehaftet, rs2: vorzeichenbehaftet, rd: obere 32 Bit des Multiplikationsergebnisses
0b010	mulhsu	$rd = (rs1 * rs2) \gg 32$ rs1: vorzeichenbehaftet, rs2: vorzeichenlos, rd: obere 32 Bit des Multiplikationsergebnisses
0b011	mulhu	$rd = (rs1 * rs2) \gg 32$ rs1: vorzeichenlos, rs2: vorzeichenlos, rd: obere 32 Bit des Multiplikationsergebnisses
0b100	div	$rd = rs1 / rs2$ rs1 und rs2 vorzeichenbehaftet
0b101	divu	$rd = rs1 / rs2$ rs1 und rs2 vorzeichenlos
0b110	rem	$rd = rs1 \% rs2$ rs1 und rs2 vorzeichenbehaftet
0b111	remu	$rd = rs1 \% rs2$ rs1 und rs2 vorzeichenlos

Tabelle 2: **Verschiedene funct3-Werte im Zusammenhang mit der M-Extension**

„*“ steht in Tabelle 2 für die Multiplikation, „/“ für die Division und „%“ für den Modulooperator.

Die eben beschriebene Schnittstelle wird durch die Signale `start_in` und `valid_out` vervollständigt. Das Signal `start_in == 1` teilt dem Modul mit, dass alle Eingänge stabil anliegen und die Berechnung gestartet werden kann. Am Ende der Berechnung wird `valid_out` auf 1 gesetzt, um mitzuteilen, dass die Berechnung abgeschlossen und das an `y_out` anliegende Ergebnis gültig ist. So können Module mit unterschiedlichen Laufzeiten in der ALU eingebunden werden, ohne Änderungen in der ALU vorzunehmen.

2.3. Integration in den Prozessorkern

Wichtig bei der Integration der M-Extension in den Prozessorkern ist, die bisherige Funktionsweise aller RV32I Instruktionen der RISC-V ISA identisch beizubehalten. Außerdem soll sich die Ausführungszeit der bisherigen Instruktionen nicht verlängern.

Wie bereits in Abschnitt 2.2 unter Abbildung 1 zu sehen, werden der Multiplizierer und Dividierer in die bestehende Struktur der ALU eingebunden.

Die Eingangssignale `a_in`, `b_in` und `funct3_in` an beiden Modulen werden immer fest angeschlossen, direkt an die gleichnamigen Eingangssignale der ALU. Um die beiden `start_in` Signale zu bilden, muss man sich erst überlegen, in welchem Fall der Multiplizierer oder Dividierer eine Berechnung durchführen soll. Bei ersterem muss festgestellt werden, ob eine Multiplikation (`mul`, `mulh`, `mulhu`, `mulhsu`) durchgeführt werden soll, bei letzterem, ob eine Division oder Restberechnung (`div`, `divu`, `rem`, `remu`) durchgeführt werden soll.

Beim erneuten Blick in Abbildung 3 sieht man, wie auch schon in Abschnitt 2.2 beschrieben, dass die Art der Berechnung der Instruktion durch die Felder `funct7`, `funct3` und `opcode` definiert wird. Das Decodieren des `opcode` übernimmt (bereits auch vor Implementation der M-Extension) das Steuerwerk, welches mit dem Signal `alu_mode == ALU_MODE_REG_REG` der ALU mitteilt, dass es sich um eine R-Type Instruktion handelt. Nun müssen nur noch `funct7` und `funct3` überprüft werden. Wie vorher erwähnt, gilt bei Instruktionen der M-Extension `funct7 == 0x1`. Bei Multiplikationen ist der höchstwertigste Bit von `funct3 == 0`, ansonsten 1.

Die Gleichung für das Multiplikationsmodul lautet also, in SystemC ausgedrückt:

```
1 mul_start = (alu_mode == ALU_MODE_REG_REG)
2           && (funct7_in == 0x1)
3           && !funct3_in[2]
```

und für das Divisionsmodul:

```
1 div_start = (alu_mode == ALU_MODE_REG_REG)
2           && (funct7_in == 0x1)
3           && funct3_in[2]
```

Nun muss noch das Berechnungsergebnis `y_out` der jeweiligen Module (`mul_result` und `div_result`) an das `y_out` der ALU ausgeleitet werden. Hierzu dient vereinfacht

betrachtet ein Multiplexer, der bei Instruktionen, die nicht Teil der M-Extension sind, das Ergebnis der bisherigen Berechnung der ALU ausgibt (somit wird das bisherige Verhalten nicht verändert), bei `mul_start == 1` ist `y_out = mul_result` und bei `div_start == 1` ist `y_out = div_result`.

Das `valid_out` Signal der beiden Berechnungsmodule muss ebenfalls berücksichtigt werden. Dieses wird einfach nach außen geleitet, ähnlich wie `y_out`. Also erhält die ALU nun auch ein neues Ausgangssignal namens `valid_out`. Bei `mul_start == 1` ist `valid_out = mul_valid`, bei `div_start == 1` ist `valid_out = div_valid` und ansonsten gilt `valid_out = 1`, da in diesem Fall rein kombinatorische Berechnungen stattfinden und das Ergebnis sofort valide ist. Dieses neue `valid_out` Signal der ALU wird nun zum Steuerwerk geführt.

Das Steuerwerk muss das neu eingeführte `valid_out` Signal (Innerhalb des Steuerwerks: `alu_valid`) berücksichtigen. Dafür kann man ein stark vereinfachtes Zustandsübergangsdiagramm des Steuerwerks ohne Interrupts, Debugger, nur mit Ausführung von R-Type Instruktionen und einer Vereinfachung des Laden des nächsten Befehls betrachten.

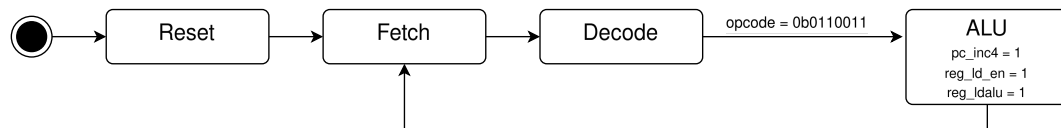


Abbildung 4: Zustandsübergangsdiagramm des Steuerwerks ohne M-Extension

Wie in Abbildung 4 zu sehen, gibt es vier Zustände im vereinfachten Steuerwerk. Nach dem *Reset* springt das Steuerwerk in den *Fetch*-Zustand, in dem der Prozessor die nächste Instruktion lädt. Anschließend wird im *Decode*-Zustand festgestellt, um welchen Typ an Instruktion es sich handelt. Wie schon erwähnt, wird hier nur die R-Type Instruktion abgebildet. Ohne M-Extension erfolgt die Berechnung der ALU rein kombinatorisch. Der Übergang vom Zustand *ALU* zurück zum Zustand *Fetch* geschieht daher nach genau einem Taktzyklus. Innerhalb des *ALU*-Zustands wird nicht nur das Ergebnis berechnet, sondern auch das Signal `pc_inc4` auf 1 gesetzt, um den Programmzähler (PC) für die nächste Instruktion zu inkrementieren. Dem Registerspeicher (Regfile) wird mittels `reg_ld_en = 1` mitgeteilt, dass der angelegte Wert in das in der Instruktion angegebene Register `rd` (siehe Abbildung 3) geschrieben werden soll. Der am Registerspeicher zum Speichern angelegte Wert soll in diesem Fall `y_out` der ALU sein. Um Daten aus unterschiedlichen Quellen in Register zu speichern, ist im Nucleus ein Multiplexer vor den Dateneingang des Registerspeichers geschaltet. Mit dem Signal `reg_ldalu` des Steuerwerks wird an diesem Multiplexer das Ergebnis der ALU aus `y_out` an den Eingang des Registerspeichers gelegt. Dieser Ablauf setzt voraus, dass alle Operationen der ALU innerhalb eines einzigen Taktzyklus abgeschlossen sind und das Ergebnis stabil am Ausgang anliegt.

Mit der Einführung der M-Extension muss dieses Prinzip aufgebrochen werden, da die Berechnung von Multiplikationen und Divisionen in Hardware gegebenenfalls länger als einen Takt dauern können. Das Signal `alu_valid` dient hierbei als Indikator für die Fertigstellung: Das Steuerwerk verweilt so lange im aktuellen Zustand, bis das Signal

auf eine erfolgreiche Berechnung hinweist, wodurch die variable Berechnungsdauer der Hardware-Module berücksichtigt wird.

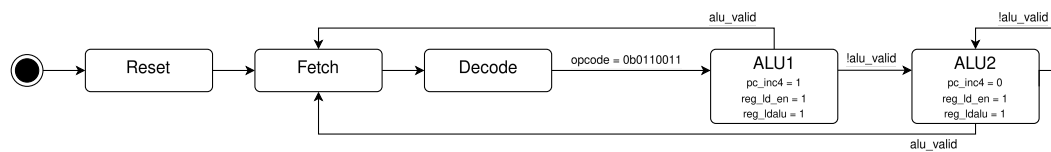


Abbildung 5: **Zustandsübergangsdiagramm des Steuerwerks mit M-Extension**

In Abbildung 5 wird das abgeänderte Zustandsübergangsdiagramm des Steuerwerks dargestellt. Zu sehen ist, dass nicht nur das Warten auf `alu_valid` hinzugefügt wurde, sondern auch zwei neue Zustände *ALU1* und *ALU2*, welche den Zustand *ALU* ersetzen. In *ALU1* und *ALU2* werden die selben Steuersignale gesetzt, wie vorher im Zustand *ALU*, mit Ausnahme von `pc_inc4`.

Im ersten Schritt nach dem *Decode* wechselt das Steuerwerk in den *ALU1*-Zustand. Hier wird das Signal `pc_inc4` auf 1 gesetzt, um den Programmzähler zu inkrementieren. Handelt es sich um eine kombinatorische Instruktion der Basis-ISA (RV32I), ist `alu_valid` sofort 1 und das Steuerwerk kehrt direkt in den *Fetch*-Zustand zurück.

Dauert die Berechnung jedoch länger, wie bei der Multiplikation oder Division, signalisiert die ALU über `!alu_valid` bzw. `alu_valid == 0` mit, dass die Berechnung noch andauert. In diesem Fall wechselt das Steuerwerk in den Zustand *ALU2*. Dieser dient als reine Warteschleife, in der `pc_inc4` wieder auf 0 gesetzt wird. So wird sichergestellt, dass auch bei längerer Berechnungsdauer der Programmzähler nur einmal inkrementiert wird. Das Steuerwerk verbleibt so lange in dieser Schleife, bis die Berechnung abgeschlossen ist (`alu_valid == 1`), wonach es wieder in den *Fetch*-Zustand zurückkehrt.

Die Signale `reg_ldalu` und `reg_ld_en` werden hier in beiden Zuständen auf 1 gesetzt, was bedeutet, dass kontinuierlich das Ergebnis der ALU (`y_out`) in das Register `rd` der Instruktion geschrieben wird. Da dem Steuerwerk nicht bekannt ist, wie lange die Berechnung dauert, können diese Signale nicht am Ende der Berechnung gesetzt werden, ohne den Prozessorzyklus um einen Takt zu verlängern, was die Performanz des PicoNut-Prozessors beeinträchtigen würde. Das dauerhafte Setzen der Signale stellt kein Problem dar. Es werden zwar während der andauernden Berechnung unvorhersehbare bzw. inkorrekte Ergebnisse aus der ALU in das Register `rd` geschrieben, jedoch ist im letzten Berechnungstakt (`alu_valid` ändert sich von 0 auf 1) das am Ausgang der ALU liegende Ergebnis korrekt und wird durch die gesetzten Signale `reg_ldalu = 1` und `reg_ld_en = 1` ebenfalls in das Register `rd` geschrieben, wodurch schlussendlich das korrekte Ergebnis in `rd` vorliegt.

2.4. Implementierung des Multiplikations- und Divisionsmoduls

In diesem Kapitel wird die Implementierung der Multiplikations- und Divisionsmodule beschrieben. Da der Schwerpunkt dieser Arbeit auf der korrekten Integration der M-Extension in das Gesamtsystem und das Steuerwerk liegt, wurde bewusst ein technisch weniger komplexer Ansatz für die Berechnungsverfahren gewählt. Dieses Kapitel bezieht sich auf das Werk *Computer Arithmetic Algorithms* von Israel Koren [2]. Die dort beschriebenen sequenziellen Verfahren *Shift-and-Add* sowie *Shift-and-Subtract* kommen

für die Multiplikation und Division zum Einsatz. Diese Entscheidung ermöglicht eine übersichtliche Verifikation der Schnittstellenlogik und hält die Architektur des PicoNut für zukünftige Optimierungen flexibel: Die gewählte modulare Struktur erlaubt es, die einfachen sequenziellen Rechenwerke zu einem späteren Zeitpunkt gegen effizientere Implementierungen auszutauschen, ohne die bereits implementierte Steuerlogik anpassen zu müssen. Zusätzlich wurde die Implementierung so gestaltet, dass sie über C-Präprozessor-Makros konfigurierbar ist. Dies erlaubt es, die M-Extension je nach Ressourcenbudget des Zielsystems vollständig einzubinden, wegzulassen oder lediglich die Zmmul-Extension einzubinden, welche ausschließlich die Multiplikationsinstruktionen bereitstellt. [1]

Im Folgenden wird zunächst das Multiplikationsmodul im Detail betrachtet, da dieses als erste funktionale Einheit der M-Extension realisiert wurde und die Grundlage für das Verständnis der sequenziellen Abläufe bildet.

Wie bereits erwähnt, kommt der Shift-and-add-Algorithmus zur Multiplikation zum Einsatz. Die Berechnung erfolgt analog zur schriftlichen Multiplikation aus der Schule, nur in diesem Fall nicht mit Zahlen aus dem Dezimalsystem, sondern mit Binärzahlen. In Abbildung 6 wird dieser Prozess beispielhaft anhand der Multiplikation zweier 4-Bit-Zahlen $a = 0b1011$ und $b = 0b0101$ visualisiert.

$$\begin{array}{r} 1\ 0\ 1\ 1 = a \\ \times 0\ 1\ 0\ 1 = b \\ \hline 0\ 1\ 0\ 1 = (0101 \ll 0) \times 1 \\ 0\ 1\ 0\ 1 = (0101 \ll 1) \times 1 \\ 0\ 0\ 0\ 0 = (0101 \ll 2) \times 0 \\ + 0\ 1\ 0\ 1 = (0101 \ll 3) \times 1 \\ \hline 0\ 0\ 1\ 1\ 0\ 1\ 1\ 1 = y \end{array}$$

Abbildung 6: **Beispiel der Shift-and-add Berechnung nach Koren [2]**

Wie in Abbildung 6 dargestellt, wird für jedes Bit des Multiplikators a geprüft, ob eine Addition des Multiplikanden b notwendig ist. Ist bei a am Bit der Stelle i eine 1, wird das um i Bits nach links geschobene b zum Gesamtergebnis addiert, ansonsten nicht.

In diesem Beispiel kommt man also auf:

$$0b1011 \times 0b0101 = 0b0101 + 0b01010 + 0b0101000 = 0b00110111$$

Das gezeigte Verfahren geht davon aus, dass mit vorzeichenlosen Zahlen gerechnet wird. Falls zwei Zahlen mit Vorzeichen multipliziert werden sollen, wird zunächst das Vorzeichen des Ergebnisses festgestellt. Eine Binärzahl ist negativ, wenn das höchstwertige Bit der jeweiligen Zahl 1 ist, ansonsten ist sie positiv. Falls beide Zahlen jeweils positiv oder negativ sind, ergibt sich ein positives Ergebnis, bei unterschiedlichen Vorzeichen ein negatives Ergebnis. Diese Logik lässt sich mittels eines einfachen XOR-Gatters realisieren, welches die höchstwertigen Bits aus den Eingangszahlen a und b als Eingang erhält. Der Ausgang des Gatters zeigt dann mit 1 ein negatives Ergebnis, mit 0 ein positives an. Wenn nur der Multiplikator als vorzeichenbehaftete Zahl und der Multiplikand als

vorzeichenlose Zahl interpretiert werden soll, hat das Ergebnis das Vorzeichen des Multiplikators, also lässt sich hier direkt aus dem höchstwertigen Bit von a das Vorzeichen des Ergebnisses ableiten.

Die eigentliche Multiplikation von Zahlen mit Vorzeichen verläuft genau wie die Multiplikation mit vorzeichenlosen Zahlen, mit einem entscheidenden Unterschied. Um die Berechnung durchzuführen, wird der Betrag der Zahl gebildet, um mit rein positiven Zahlen zu multiplizieren. Falls die Zahl negativ ist, muss das Zweierkomplement gebildet werden. Hierfür muss nur die jeweilige Zahl invertiert (alle Bits werden negiert; aus 1 wird 0 und umgekehrt) und anschließend eine 1 addiert werden. Im Fall einer positiven Zahl passiert nichts. Diese Verhaltensweisen bilden die Instruktionen `mulh`, `mulhu` und `mulhsu` ab, wo jeweils zwei vorzeichenbehaftete, zwei vorzeichenlose bzw. eine vorzeichenbehaftete und eine vorzeichenlose Zahl multipliziert werden.

Nach der Berechnung liegt nun das vorzeichenlose (positive) Ergebnis vor. Dieses muss nur noch, falls das Ergebnis negativ sein soll, mittels des Zweierkomplements in eine negative Zahl überführt werden.

Die Bitbreite des Multiplikationsergebnisses ist das doppelte der Bitbreite der jeweiligen Eingangswerte. Beispielsweise kann man sich die Multiplikation mit zwei maximal großen 4-Bit-Zahlen betrachten: $0b1111 \times 0b1111 = 0b1110\ 0001$. Im Fall des PicoNut-Prozessors, welcher mit 32-Bit-Registern arbeitet, wird ein 64-Bit großes Multiplikationsergebnis generiert.

In der Hardware-Umsetzung wird dieses Schema jedoch nicht rein kombinatorisch, sondern sequenziell abgearbeitet. Anstatt alle Teilprodukte gleichzeitig zu berechnen und zusammenzuaddieren, wird ein 64-Bit großes Register verwendet, welches das Zwischenergebnis speichert und zu diesem in jedem Takt das neue Teilergebnis addiert wird. Im eben genannten Beispiel kommt man mit diesem Aufbau auf vier Takte zur Berechnung, im Fall des PicoNut sind es 32 Takte, da mit 32-Bit breiten Eingangswerten gerechnet wird.

Die sequenzielle Abarbeitung lässt sich in drei Zustände aufteilen, die in Abbildung 7 dargestellt sind.

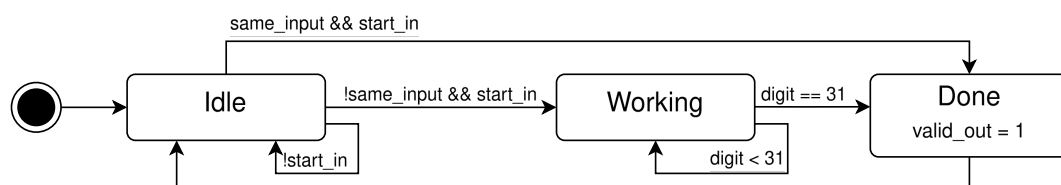


Abbildung 7: **Zustandsübergangsdiagramm des Multiplikationsmoduls**

Im *Idle*-Zustand wartet das Modul auf `start_in == 1`, um mit der Berechnung zu beginnen. Beim Übergang in den *Working*-Zustand wird der interne Zähler `digit` sowie das Register des Zwischenergebnisses auf 0 zurückgesetzt. Der Zähler `digit` dient als Iterationsindex, steht also für die Stelle des aktuell zu überprüfenden Bits im Multiplikator und bestimmt gleichzeitig den Verschiebewert für das Teilprodukt. Mit jedem Taktzyklus im *Working*-Zustand wird `digit` inkrementiert, bis nach 32 Durchläufen (`digit == 31`) die vollständige Bitbreite abgearbeitet wurde. Nach Abschluss der Berechnung wechselt

das Modul in den *Done*-Zustand, wobei einerseits `valid_out = 1` gesetzt wird, um dem Steuerwerk die abgeschlossene Berechnung zu signalisieren. Andererseits muss `y_out`, also das Ausgangsergebnis des Moduls, korrekt belegt werden.

Multiplikator und Multiplikand werden mittels Registern zwischengespeichert, wobei die gespeicherten Werte beim Verlassen des Idle-Zustands auf die am Eingang anliegenden aktualisiert werden. Das modulinterne Signal `same_input` prüft, ob die an beiden Eingängen anliegenden Werte identisch zur letzten Berechnung sind. Falls ja (`same_input == 1`), kann direkt in den Done-Zustand gewechselt werden, wobei das noch im Ergebnisregister vorhandene Rechenergebnis direkt ausgegeben werden kann. Dies ist nützlich, wenn die gesamten 64 Bit des Rechenergebnisses benötigt werden. Hier können im Programmcode direkt aufeinanderfolgende Instruktionen zur Berechnung der unteren und oberen 32 Bit der selben Zahl schneller abgearbeitet werden, da eine Berechnung übersprungen wird. Falls sich die anliegenden Werte von den zwischengespeicherten unterscheiden (`same_input == 0`), erfolgt die Berechnung regulär wie eben beschrieben.

Wie in Tabelle 2 bereits dargestellt, gibt es vier verschiedene Instruktionen zur Multiplikation. `mul` ist hierbei die einfachste Instruktion, da hier nur die untere Hälfte der Berechnung ausgegeben werden muss. In diesem Fall kann bei der Multiplikation von vorzeichenlosen Zahlen ausgegangen werden. Ein Beispiel dazu ist die Multiplikation mit 4-Bit großen Zahlen bei gleichem Vorzeichen:

$$+2 \times +3 = 0b0010 \times 0b0011 = 0b0000\ 0110 \Rightarrow 0b0110 = +6$$

$$-2 \times -3 = 0b1110 \times 0b1101 = 0b1011\ 0110 \Rightarrow 0b0110 = +6$$

Diese Funktionsweise trifft auch bei unterschiedlichen Vorzeichen zu:

$$+2 \times -3 = 0b0010 \times 0b1101 = 0b0001\ 1010 \Rightarrow 0b1010 = -6$$

$$-2 \times +3 = 0b1110 \times 0b0011 = 0b0010\ 1010 \Rightarrow 0b1010 = -6$$

Bei `mulhu` kann genauso verfahren werden, da hier die Zahlen ohne Vorzeichen interpretiert werden sollen. Für die anderen zwei Instruktionen `mulhsu` und `mulh` muss dann dementsprechend eine Vorzeichenbehandlung wie erläutert stattfinden.

Nachdem die Implementierung der Multiplikation als erste Einheit der M-Extension dargelegt wurde, befasst sich der folgende Abschnitt mit dem Divisionsmodul. Dieses baut teilweise auf der Logik des eben vorgestellten Multiplikationsmoduls auf, wobei viele Parallelen zueinander bestehen.

Zur Division wird der schon erwähnte Shift-and-subtract-Algorithmus verwendet. Wie beim Shift-and-add-Algorithmus, baut dieser ebenfalls auf der schriftlichen Division, wie man sie in der Schule lernt, auf. Abbildung 8 zeigt den Prozess am Beispiel der Berechnung mit den zwei 4-Bit-Zahlen $a = 0b1011 = 11$ als Dividend und $b = 0b0011 = 3$ als Divisor.


```

a / b = 1 0 1 1 / 0 0 1 1 = 0 0 1 1 = quotient
      -
      1
      - 0
      ---
        1 0
        - 0
        ---
          1 0 1
          - 0 0 1 1
          -----
            1 0 1
            - 0 0 1 1
            -----
              1 0 = remainder

```

Abbildung 8: **Beispiel der Shift-and-subtract Berechnung nach Koren [2]**

Die Division wird hier als schrittweise Subtraktion des Divisors vom Dividenten durchgeführt. Der Prozess geht von „links nach rechts“ durch, beginnt also an der höchstwertigen Stelle des Dividenten. In jedem Schritt wird geprüft, ob der Divisor kleiner oder gleich groß als der aktuelle Teilrest ist. Ist dies der Fall, wird an der entsprechenden Stelle des Quotienten eine 1 gesetzt und die Subtraktion durchgeführt. Ansonsten wird eine 0 im Quotienten vermerkt und der Teilrest unverändert in den nächsten Schritt übernommen (-0). Dieses Verfahren nennt sich *Restoring*-Division. Durch das fortlaufende Nachrücken der nächsten Bit-Stelle des Dividenten – in Abbildung 8 durch das „Herunterziehen“ der Ziffern dargestellt – ergibt sich nach vier Iterationen ein Quotient von $0b0011 = 3$ sowie ein verbleibender Rest (remainder) von $0b0010 = 2$.

Wie bereits beim Shift-and-add-Verfahren geht auch das eben dargestellte Shift-and-subtract-Verfahren von vorzeichenlosen Zahlen zur Berechnung aus. Die Vor- und Nachbehandlung der Vorzeichen durch Bilden des Zweierkomplement funktioniert identisch. Anders sind nur die Bedingungen dafür, wann das Ergebnis welches Vorzeichen bekommt.

Vorzeichen Dividend (a)	Vorzeichen Divisor (b)	Vorzeichen Quotient	Vorzeichen Rest
+	+	+	+
+	−	−	+
−	+	−	−
−	−	+	−

Tabelle 3: **Vorzeichenverhalten bei Division und Restberechnung**

In Tabelle 3 werden die verschiedenen Kombinationen von Vorzeichen von Dividend und Divisor gezeigt und welche Auswirkungen sie auf das Vorzeichen des Ergebnisses haben.

Dieses Verhalten wird in der RISC-V ISA [1] festgelegt. Die Spezifikation definiert auch, dass folgende Gleichung gelten muss: $\text{Dividend} = \text{Divisor} \times \text{Quotient} + \text{Rest}$.

Grenzfälle bei der Division müssen ebenfalls beachtet werden, daher werden diese ebenfalls in der Spezifikation definiert. Das Verhalten in beiden Grenzfällen für jede Instruktion werden in Tabelle 4 dargestellt.

Grenzfall	Dividend	Divisor	divu	remu	div	rem
Teilen durch Null	x	0	$2^{32} - 1$	x	-1	x
Überlauf (nur bei Berechnung mit Vorzeichen)	-2^{31}	-1	-	-	-2^{31}	0

Tabelle 4: **Grenzfallverhalten der Division nach RISC-V Foundation [1]**

Analog zur Hardware-Umsetzung der Multiplikation, wurde dieses Schema ebenfalls als sequenzielle Abarbeitung implementiert. In diesem Fall werden zwei 32-Bit große Register verwendet, um die Zwischenergebnisse für den Quotient und Rest zu speichern. Identisch zur Multiplikation benötigt die Division ebenfalls 32 Takte zur Berechnung.

Ähnlich zur Multiplikation lässt sich die sequenzielle Abarbeitung in mehrere Zustände aufteilen, wobei es bei der Division vier Zustände sind.

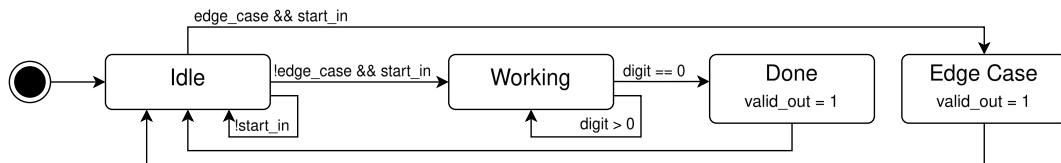


Abbildung 9: **Zustandsübergangsdiagramm des Divisionsmoduls**

Geblichen sind die Zustände *Idle*, *Working* und *Done*, neu dazu kommt der Zustand *Edge Case*. Solange $\text{start_in} = 0$ gilt, bleibt das Modul im Idle-Zustand. Falls es zu 1 wechselt, erfolgt ein Wechsel in den Zustand Working oder Edge Case, abhängig von edge_case . Das modulinterne Signal edge_case zeigt an, ob es sich um einen Grenzfall handelt. Wenn ja ($\text{edge_case} == 1$), wechselt das Modul in den Edge-Case-Zustand, in dem es das definierte Ergebnis nach Tabelle 4 an y_out ausgibt und $\text{valid_out} = 1$ setzt, um die Gültigkeit des Ergebnisses zu signalisieren. Falls kein Grenzfall vorliegt ($\text{edge_case} == 0$), ist Working der Folgezustand. Beim Übergang in diesen Zustand werden die Ergebnisregister für den Quotienten und den Rest auf 0 zurückgesetzt. Identisch zur Multiplikation ist digit der Iterationsindex der Division. Da die Division vom höchst- zum niedrigstwertigsten Bit durchgeht, beginnt digit hier bei 31 und wird in jedem Taktzyklus dekrementiert, bis nach 32 Durchläufen ($\text{digit} == 0$) die vollständige Bitbreite abgearbeitet wurde. Im Anschluss wechselt das Modul in den Zustand Done, in dem es einerseits $\text{valid_out} = 1$ setzt und andererseits abhängig von der Instruktion y_out entweder auf den berechneten Rest (bei Instruktion *rem* oder *remu*) oder Quotient (bei *div* oder *divu*) setzt. Das Vorzeichen des Ergebnisses ergibt sich aus Tabelle 3.

2.5. Testen des neuen Nucleus mit M-Extension

Um die funktionale Korrektheit des implementierten Multiplikations- und Divisionsmoduls sowie deren Integration in die ALU und den Nucleus zu überprüfen, wurden verschiedene Verifikationen durchgeführt. Der Testprozess gliederte sich in zwei wesentliche Teile: die automatisierte Validierung in Simulation durch ein offizielles Test-Framework der RISC-V Foundation und einen manuellen Performanztest auf echter Hardware.

Zunächst wurde das Test-Framework *RISCOF* (RISC-V Architectural Compliance Test Framework) [7] eingesetzt, welches für Entwickler von Hardware-Logik gedacht ist, um ihre RISC-V Implementation zu testen. Da dieses Framework bereits im PicoNut-Projekt integriert war, konnten die offiziellen Tests für die M-Extension direkt verwendet werden. Diese Tests stellen sicher, dass die Implementierung der Instruktionen korrekt nach Spezifikation erfolgt ist.

Im Anschluss folgte der Performanztest, um den praktischen Nutzen der Hardware-Beschleunigung zu quantifizieren. Hierzu wurde ein Testprogramm in C implementiert, das innerhalb zweier verschachtelter Schleifen (jeweils eine pro Operand) eine hohe Anzahl an Multiplikations-, Divisions- und Restwert-Operationen ausführt. Zum Messen der erreichten Beschleunigung durch die neu eingeführten Module der M-Extension muss dieses Programm einmal ohne und einmal mit aktivierter M-Extension ausgeführt werden. Der Messwert wird mittels der *CLINT*-Peripherie (Core Local Interruptor) festgestellt. Diese wird im Testsystem mit eingebunden und enthält unter anderem das *MTIME*-Register, welches pro Prozessortakt inkrementiert wird. Um die Anzahl der Takte, die ein Programm zur Ausführung benötigt, zu messen, muss dieses Register einmal am Anfang und einmal am Ende des Programms ausgelesen werden. Die Differenz dieser Werte ist die Anzahl an Takte, die während der Programmausführung vergangen sind.

Um die Ausführung auf Hardware zu testen, wurde das ULX3S-FPGA-Board verwendet. Der ULX3S ist ein preiswertes und komplett quelloffenes FPGA-Board, welches seit 2016 aktiv unter Kooperation vom Zagreb Makerspace Radiona.org mit der Fakultät für Elektrotechnik und Informatik der Universität Zagreb in Kroatien entwickelt wird. [8]

Der Messwert der Berechnung ohne M-Extension liegt bei 148966997 Takten, nach Aktivierung der M-Extension bei 76144155 Takten. Die Beschleunigung liegt also bei: $\frac{148966997}{76144155} \approx 1.956$. Das Nutzen der implementierten M-Extension bringt also in etwa eine Verdoppelung der Ausführungsgeschwindigkeit, also eine Halbierung der Ausführungszeit, im Fall des Beispielprogramms.

3. Fazit

3.1. Zusammenfassung

Im Rahmen dieser Projektarbeit wurde die M-Extension der RISC-V ISA [1] erfolgreich in den Nucleus des PicoNut-Prozessors integriert. Ziel war es, die Rechenleistung des Kerns durch dedizierte Hardware-Module zu steigern, während gleichzeitig die flexible Erweiterbarkeit des PicoNut weiterhin besteht, da die gewählte Umsetzung der Multiplikation und Division als abgekapselte Module erfolgte, die einfach ersetzt werden können. Außerdem wurde die Integration der M-Extension in den PicoNut umfangreich getestet, wo mittels einer Beispielanwendung ungefähr eine Halbierung der Rechenzeit durch den Einsatz der implementierten M-Extension gemessen werden konnte. Innerhalb dieser Tests konnte auch festgestellt werden, dass sich der aus dieser Arbeit vorgegangene neue Prozessorkern sowohl in Software-Simulation, als auch in Hardware korrekt verhält.

3.2. Ausblick

Durch die Modularität der gewählten Umsetzung der M-Extension können in Zukunft die rudimentären Rechenverfahren Shift-and-add zur Multiplikation und Shift-and-subtract zur Division auch gegen schnellere Algorithmen ersetzt werden, wie sie auch in Israel Korens Werk Computer Arithmetic Algorithms [2] erläutert sind.

Literatur- und Quellenverzeichnis

- [1] RISC-V Foundation, „The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20250508“. Zugegriffen: 29. Januar 2026. [Online]. Verfügbar unter: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>
- [2] I. Koren, *Computer arithmetic algorithms*, 2. Aufl. A K Peters, 2002.
- [3] „PicoNut - Forschungsgruppe Effiziente Eingebettete Systeme“. Zugegriffen: 22. Januar 2026. [Online]. Verfügbar unter: <https://ees.tha.de/piconut/index.html>
- [4] „IEEE Standard SystemC(R) Language Reference Manual“, *IEEE Std 1666-2005*, Bd. , S. 1–423, 2006, doi: [10.1109/IEEESTD.2006.99475](https://doi.org/10.1109/IEEESTD.2006.99475).
- [5] „Typst Documentation“. Zugegriffen: 22. Januar 2026. [Online]. Verfügbar unter: <https://typst.app/docs/>
- [6] „PicoNut Documentation (CC-BY 4.0, <https://creativecommons.org/licenses/by/4.0/>)“. Zugegriffen: 22. Januar 2026. [Online]. Verfügbar unter: <https://ees.tha.de/piconut/manual/>
- [7] „RISC-V Architectural Testing Framework - README.adoc“. Zugegriffen: 30. Januar 2026. [Online]. Verfügbar unter: <https://github.com/riscv-non-isa/riscv-arch-test/blob/dev/doc/README.adoc>
- [8] „ULX3S Webpage“. Zugegriffen: 1. Februar 2026. [Online]. Verfügbar unter: <https://radiona.org/ulx3s/>